WEEK 4 TASK: MACHINE LEARNING MODEL DEVELOPMENT AND EVALUATION

Project Title: Milk Quality Prediction Using Machine Learning

Student Name: Diksha Deshpande

Domain: Data Science

Tools Used: Python, Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, Google Colab

1. Introduction

Machine Learning enables computers to learn patterns from data and make predictions on unseen observations. In this task, a machine learning classification model was developed using the Milk Quality Dataset. The complete process included data preparation, feature selection, model training, prediction, evaluation, and interpretation.

2. Objectives

1. To prepare the dataset for machine learning.
2. To select suitable features and target variables.
3. To split the dataset into training and testing data.
4. To develop classification models using Scikit-learn.
5. To evaluate model performance using appropriate metrics.
6. To identify potential sources of prediction errors.
7. To suggest possible improvements to the model.

3. Dataset

The Milk Quality Dataset contains pH, Temprature, Taste, Odor, Fat, Turbidity, Colour, and Grade.

The variables pH, Temprature, Taste, Odor, Fat, Turbidity, and Colour were used as input features, while Grade was used as the target variable for classification.

4. Data Preparation

The dataset was inspected and prepared before model development. Missing values, duplicate records, data types, and feature structure were checked.

The target variable was separated from the input features, and the dataset was divided into training and testing subsets.

The training data was used to train the models, while the testing data was used to evaluate their performance on unseen observations.

5. Model Selection

Classification algorithms were selected because the target variable, Grade, represents different milk quality categories.

Decision Tree and Random Forest classification models were considered for the analysis.

Decision Tree

A Decision Tree was selected because it is simple to understand and can represent decision-making rules based on input features.

Random Forest

Random Forest was selected because it combines multiple decision trees and can provide more robust predictions than a single tree.

6. Model Training

The selected models were trained using the training dataset. After training, predictions were generated using the testing dataset.

The complete Python implementation is provided in the attached Code and Output PDF.

7. Model Evaluation

The trained models were evaluated using classification metrics such as:

Accuracy

Precision

Recall

F1-Score

Confusion Matrix

These metrics help determine how effectively the models classify milk quality grades.

8. Model Performance Comparison

The performance of the developed models was compared using their evaluation metrics.

The model showing the strongest overall performance was considered the preferred model for the classification task.

9. Visualization 1 – Confusion Matrix

A confusion matrix was used to compare the actual milk quality grades with the grades predicted by the model.

The confusion matrix helps identify correctly classified observations as well as misclassified observations for each quality category.

10. Visualization 2 – Model Performance Comparison

A comparison visualization was used to compare the performance metrics of the developed machine learning models.

This visualization provides an easy way to identify which model performed better across the selected evaluation metrics.

11. Results

The machine learning analysis demonstrated that milk quality parameters can be used to classify milk quality grades.

The actual accuracy, precision, recall, and F1-score obtained during the analysis were used to compare the models.

The best-performing model was selected based on its overall evaluation performance.

12. Sources of Error

Possible sources of prediction errors include:

1. Limited dataset size.
2. Imbalanced representation of quality grades.
3. Measurement errors in input parameters.
4. Limited number of features.
5. Variations in milk samples and storage conditions.
6. Overfitting or underfitting during model training.
7. Differences between the available dataset and real-world data.

13. Overfitting and Underfitting

Overfitting occurs when a model learns the training data too closely and performs poorly on unseen data. Underfitting occurs when a model is too simple to capture important patterns in the data.

Comparing training and testing performance can help identify potential overfitting or underfitting.

14. Suggestions for Model Improvement

The model can be improved by:

1. Collecting a larger and more diverse dataset.
2. Including additional milk quality parameters.
3. Applying feature selection and preprocessing techniques.
4. Using cross-validation.
5. Performing hyperparameter tuning.
6. Testing additional machine learning algorithms.
7. Handling class imbalance where necessary.
8. Validating the model using an independent real-world dataset.

15. Strengths of the Model

The developed approach provides a systematic method for classifying milk quality based on measurable parameters. Machine learning can process multiple input characteristics simultaneously and provide a preliminary quality prediction.

16. Limitations

The model is dependent on the quality and representativeness of the available dataset. Its performance may change when applied to new data from different regions, suppliers, seasons, or testing environments.

The model should therefore be considered a decision-support tool rather than a complete replacement for laboratory milk-quality testing.

17. Business Impact

A machine learning-based milk quality classification system could support dairy quality-control processes by providing rapid preliminary assessment of milk samples.

It may help identify samples requiring additional testing, support consistent quality monitoring, and contribute to data-driven decision-making.

18. Conclusion

The Week 4 task successfully demonstrated the development and evaluation of machine learning classification models using the Milk Quality Dataset.

The process included data preparation, feature and target selection, model training, prediction, and evaluation. Decision Tree and Random Forest approaches were used to classify milk quality grades.

The evaluation metrics and visualizations provided a basis for comparing model performance and identifying potential sources of error. With a larger dataset, additional features, hyperparameter tuning, and real-world validation, the system could be further improved.

19. Code and Visualization Outputs

The complete Python code used for data preparation, model training, prediction, evaluation, and visualization, along with all generated outputs, is provided in the attached PDF.

Appendix A: Complete Python Code, Model Outputs, and Visualizations

20. References

1. Python Documentation.
2. Pandas Documentation.
3. NumPy Documentation.
4. Scikit-learn Documentation.
5. Matplotlib Documentation.
6. Seaborn Documentation.
7. Milk Quality Dataset.

In [28…
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

In [43…
```
print(df.head())
print(df.shape)
print(df.info())
print(df.isnull().sum())
```

```
    pH  Temprature  Taste  Odor  Fat   Turbidity  Colour   Grade
0  6.6          35      1     0     1          0     254    high
1  6.6          36      0     1     0          1     253    high
2  8.5          70      1     1     1          1     246     low
3  9.5          34      1     1     0          1     255     low
4  6.6          37      0     0     0          0     255  medium
(1059, 8)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1059 entries, 0 to 1058
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   pH          1059 non-null   float64
 1   Temprature  1059 non-null   int64
 2   Taste       1059 non-null   int64
 3   Odor        1059 non-null   int64
 4   Fat         1059 non-null   int64
 5   Turbidity   1059 non-null   int64
 6   Colour      1059 non-null   int64
 7   Grade       1059 non-null   object
dtypes: float64(1), int64(6), object(1)
memory usage: 66.3+ KB
None
pH            0
Temprature    0
Taste         0
Odor          0
Fat           0
Turbidity     0
Colour        0
Grade         0
dtype: int64
```

In [44…
```
X = df.drop('Grade', axis=1)
y = df['Grade']

print("Features:")
print(X.head())

print("\nTarget:")
print(y.head())
```

```
print("\nFeature shape:", X.shape)
print("Target shape:", y.shape)
```

```
Features:
    pH  Temprature  Taste  Odor  Fat   Turbidity  Colour
0  6.6          35      1     0     1          0     254
1  6.6          36      0     1     0          1     253
2  8.5          70      1     1     1          1     246
3  9.5          34      1     1     0          1     255
4  6.6          37      0     0     0          0     255

Target:
0      high
1      high
2       low
3       low
4    medium
Name: Grade, dtype: object

Feature shape: (1059, 7)
Target shape: (1059,)
```

In [45…
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (847, 7)
Testing data: (212, 7)
```

In [46…
```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    random_state=42,
    max_depth=5
)

model.fit(X_train, y_train)

print("Decision Tree model trained successfully!")
```

```
Decision Tree model trained successfully!
```

In [47…
```
y_pred = model.predict(X_test)

print("Actual values:")
print(y_test.values)
```

```
print("\nPredicted values:")
print(y_pred)
```

```
Actual values:
['medium' 'medium' 'medium' 'high' 'low' 'low' 'low' 'medium' 'low' 'hig
h'
 'medium' 'low' 'medium' 'high' 'medium' 'low' 'medium' 'high' 'low'
 'medium' 'high' 'high' 'high' 'medium' 'medium' 'medium' 'medium' 'high'
 'medium' 'high' 'low' 'low' 'low' 'medium' 'low' 'low' 'medium' 'medium'
 'high' 'medium' 'medium' 'medium' 'high' 'high' 'low' 'low' 'low' 'high'
 'high' 'low' 'high' 'medium' 'low' 'medium' 'low' 'low' 'high' 'low'
 'medium' 'medium' 'medium' 'medium' 'high' 'medium' 'low' 'low' 'low'
 'high' 'low' 'medium' 'high' 'low' 'low' 'medium' 'medium' 'high'
 'medium' 'low' 'low' 'high' 'medium' 'low' 'low' 'low' 'low' 'high' 'lo
w'
 'high' 'low' 'low' 'low' 'low' 'medium' 'medium' 'low' 'low' 'low'
 'medium' 'high' 'low' 'medium' 'medium' 'medium' 'high' 'medium' 'high'
 'low' 'high' 'low' 'low' 'low' 'medium' 'low' 'medium' 'medium' 'medium'
 'high' 'low' 'medium' 'high' 'low' 'medium' 'low' 'low' 'low' 'low'
 'high' 'medium' 'low' 'medium' 'low' 'low' 'low' 'low' 'low' 'medium'
 'medium' 'low' 'medium' 'low' 'medium' 'medium' 'low' 'low' 'high' 'hig
h'
 'medium' 'high' 'low' 'high' 'medium' 'high' 'low' 'low' 'high' 'low'
 'medium' 'high' 'high' 'medium' 'high' 'low' 'low' 'low' 'medium' 'low'
 'low' 'low' 'medium' 'low' 'high' 'low' 'high' 'medium' 'medium' 'mediu
m'
 'low' 'low' 'medium' 'medium' 'low' 'low' 'medium' 'medium' 'high' 'hig
h'
 'low' 'high' 'low' 'low' 'low' 'medium' 'low' 'low' 'high' 'low' 'mediu
m'
 'high' 'medium' 'high' 'high' 'medium' 'high' 'medium' 'medium' 'medium'
 'medium' 'medium' 'high' 'medium' 'low' 'high']

Predicted values:
['medium' 'medium' 'medium' 'high' 'low' 'low' 'low' 'medium' 'low' 'hig
h'
 'medium' 'low' 'medium' 'high' 'medium' 'low' 'medium' 'medium' 'low'
 'medium' 'high' 'medium' 'medium' 'medium' 'medium' 'medium' 'medium'
 'high' 'medium' 'high' 'low' 'low' 'low' 'medium' 'low' 'low' 'medium'
 'medium' 'medium' 'medium' 'medium' 'medium' 'high' 'medium' 'low' 'low'
 'low' 'high' 'medium' 'low' 'medium' 'medium' 'low' 'medium' 'low' 'low'
 'medium' 'low' 'medium' 'medium' 'medium' 'medium' 'medium' 'medium'
 'low' 'low' 'low' 'high' 'low' 'medium' 'high' 'low' 'low' 'medium'
 'medium' 'high' 'medium' 'low' 'low' 'high' 'medium' 'low' 'low' 'low'
 'low' 'high' 'low' 'high' 'low' 'low' 'low' 'low' 'medium' 'medium' 'lo
w'
 'low' 'low' 'medium' 'medium' 'low' 'medium' 'medium' 'medium' 'high'
 'medium' 'medium' 'low' 'medium' 'low' 'low' 'low' 'medium' 'low'
 'medium' 'medium' 'medium' 'high' 'low' 'medium' 'medium' 'low' 'medium'
 'low' 'high' 'low' 'low' 'high' 'medium' 'low' 'medium' 'low' 'low' 'lo
w'
 'low' 'low' 'medium' 'medium' 'low' 'medium' 'low' 'medium' 'medium'
 'low' 'low' 'high' 'high' 'medium' 'medium' 'low' 'medium' 'medium'
 'medium' 'low' 'low' 'high' 'low' 'medium' 'medium' 'medium' 'medium'
 'high' 'low' 'low' 'low' 'medium' 'low' 'low' 'low' 'medium' 'low' 'hig
h'
 'low' 'high' 'medium' 'medium' 'medium' 'low' 'low' 'medium' 'medium'
 'low' 'low' 'medium' 'medium' 'medium' 'medium' 'low' 'high' 'low' 'low'
```

```
 'low' 'medium' 'low' 'low' 'medium' 'low' 'medium' 'medium' 'medium'
 'high' 'medium' 'medium' 'medium' 'medium' 'medium' 'medium' 'medium'
 'medium' 'high' 'medium' 'low' 'high']
```

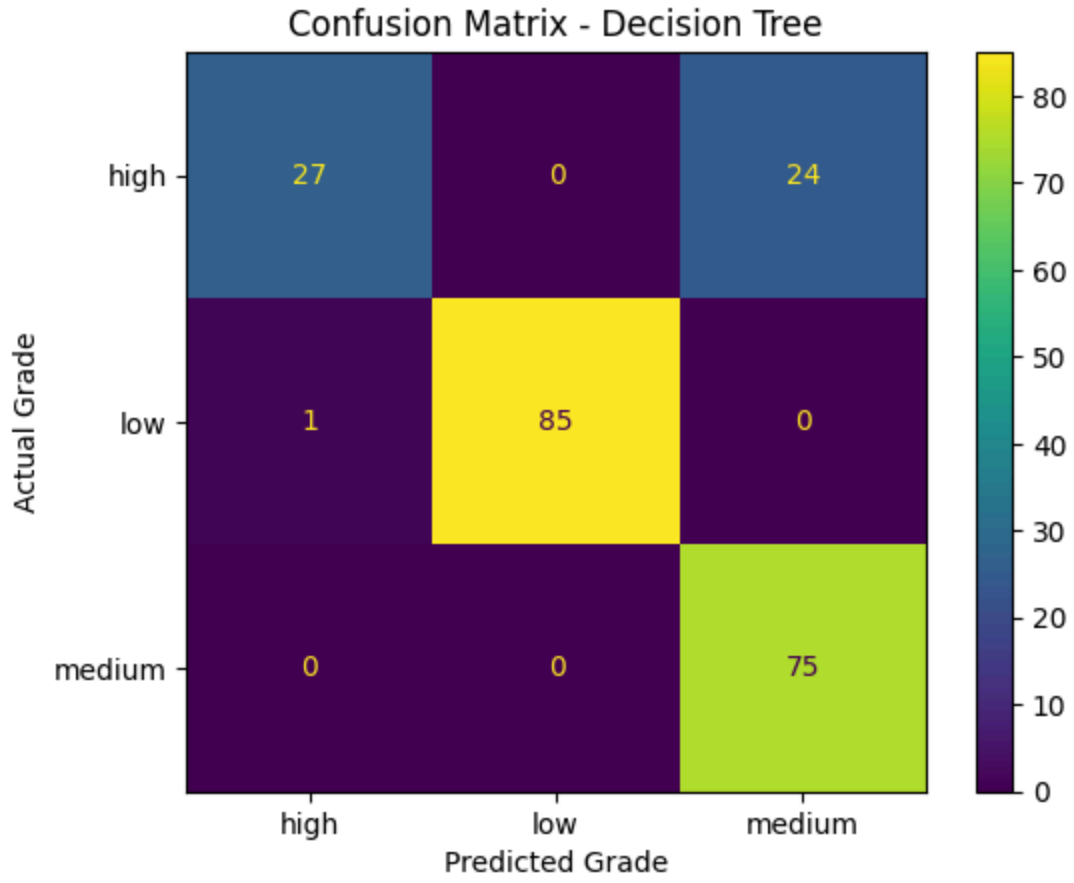
In [50…

```python
from sklearn.metrics import accuracy_score, precision_score, recall_score

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1-Score :", f1)
```

```
Accuracy : 0.8820754716981132
Precision: 0.9056450624846851
Recall   : 0.8820754716981132
F1-Score : 0.8727028675983763
```

In [49…

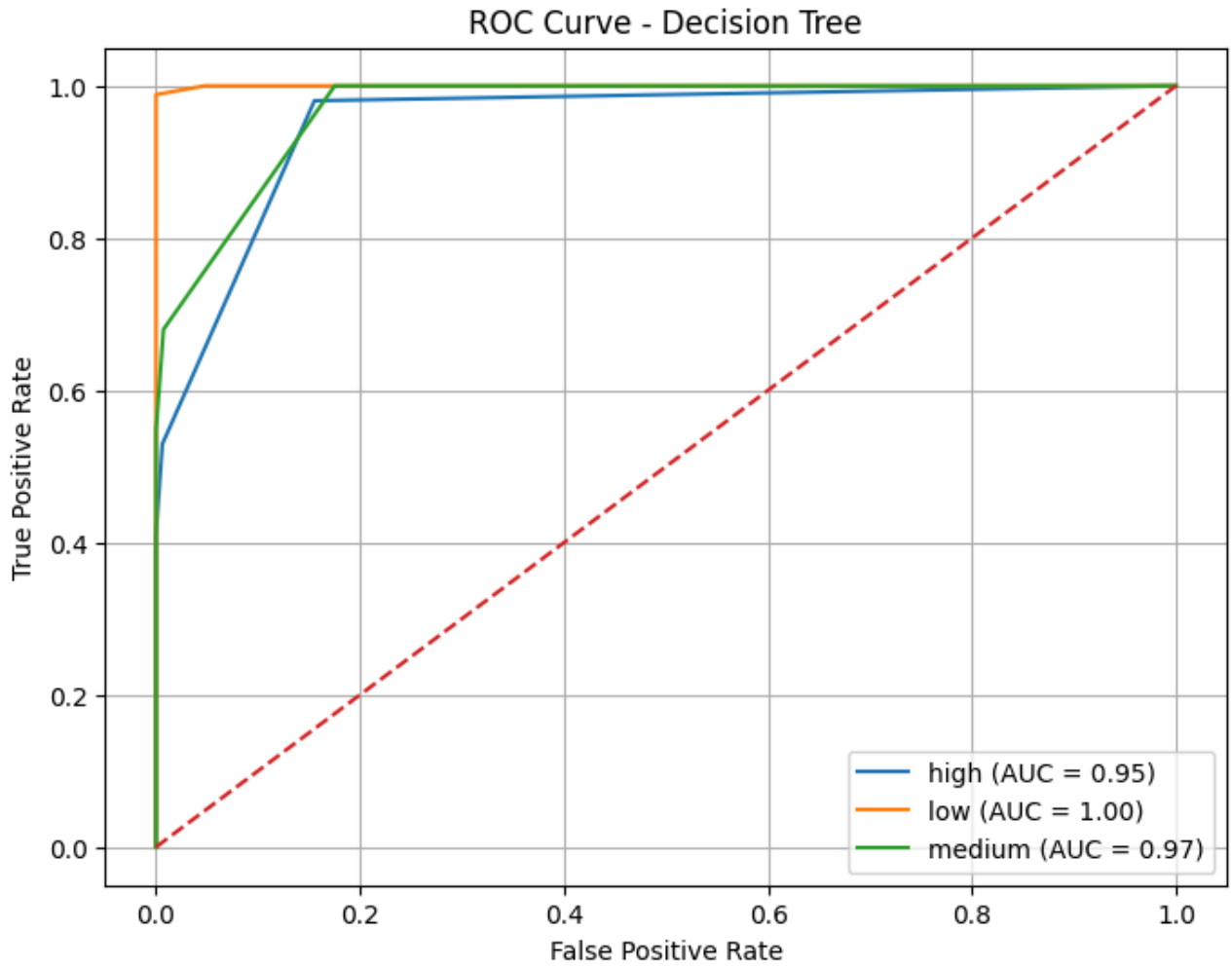
```python
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=model.classes_
)

disp.plot()
plt.title("Confusion Matrix - Decision Tree")
plt.xlabel("Predicted Grade")
plt.ylabel("Actual Grade")
plt.show()
```



In [51…

```python
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize
import matplotlib.pyplot as plt

# Convert target labels into binary/multiclass format
classes = model.classes_
y_test_bin = label_binarize(y_test, classes=classes)

# Get prediction probabilities
y_prob = model.predict_proba(X_test)

plt.figure(figsize=(8, 6))

for i, class_name in enumerate(classes):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_prob[:, i])
    roc_auc = auc(fpr, tpr)

    plt.plot(
        fpr,
        tpr,
        label=f"{class_name} (AUC = {roc_auc:.2f})"
    )

plt.plot([0, 1], [0, 1], linestyle="--")
```

```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Decision Tree")
plt.legend()
plt.grid()
plt.show()
```



In [52…

```
from sklearn.metrics import classification_report

print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

        high       0.96      0.53      0.68        51
         low       1.00      0.99      0.99        86
      medium       0.76      1.00      0.86        75

    accuracy                           0.88       212
   macro avg       0.91      0.84      0.85       212
weighted avg       0.91      0.88      0.87       212
```

In [53…

```python
errors = (y_test != y_pred).sum()

print("Total test samples:", len(y_test))
print("Incorrect predictions:", errors)
print("Error rate:", errors / len(y_test))
```

```
Total test samples: 212
Incorrect predictions: 25
Error rate: 0.1179245283018868
```

In [54…

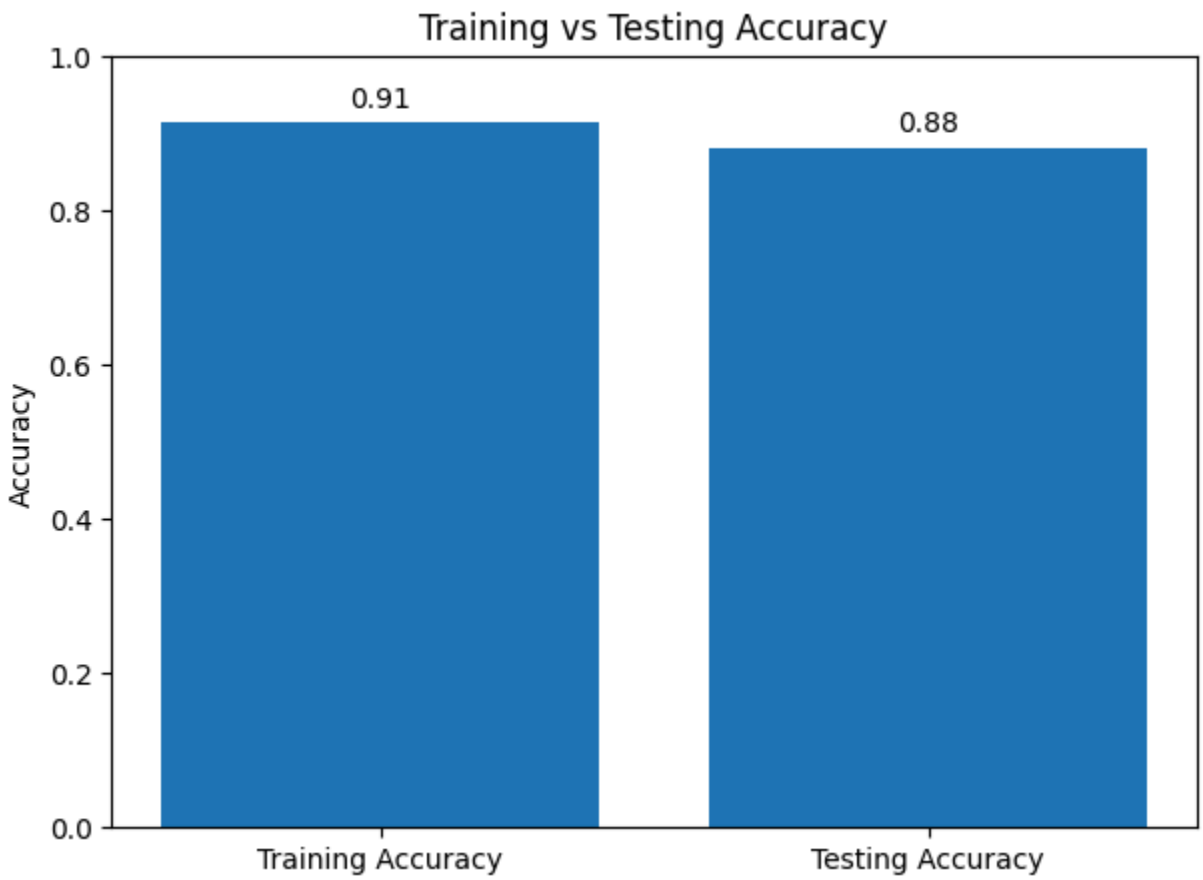
```python
from sklearn.metrics import accuracy_score

train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

train_accuracy = accuracy_score(y_train, train_pred)
test_accuracy = accuracy_score(y_test, test_pred)

print("Training Accuracy:", train_accuracy)
print("Testing Accuracy :", test_accuracy)

print("\nDifference:", train_accuracy - test_accuracy)
```

```
Training Accuracy: 0.9138134592680047
Testing Accuracy : 0.8820754716981132

Difference: 0.031737987569891546
```

In [55…

```python
import matplotlib.pyplot as plt

accuracies = [train_accuracy, test_accuracy]
labels = ["Training Accuracy", "Testing Accuracy"]

plt.figure(figsize=(7, 5))
plt.bar(labels, accuracies)

plt.ylim(0, 1)
plt.ylabel("Accuracy")
plt.title("Training vs Testing Accuracy")

for i, value in enumerate(accuracies):
    plt.text(i, value + 0.02, f"{value:.2f}", ha="center")
```

```
plt.show()
```



In [56…

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

param_grid = {
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

grid_search = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy'
)

grid_search.fit(X_train, y_train)

print("Best Parameters:", grid_search.best_params_)
print("Best Cross-Validation Accuracy:", grid_search.best_score_)
```

```
Best Parameters: {'max_depth': 10, 'min_samples_leaf': 2, 'min_samples_sp
lit': 2}
Best Cross-Validation Accuracy: 0.9917229376957885
```

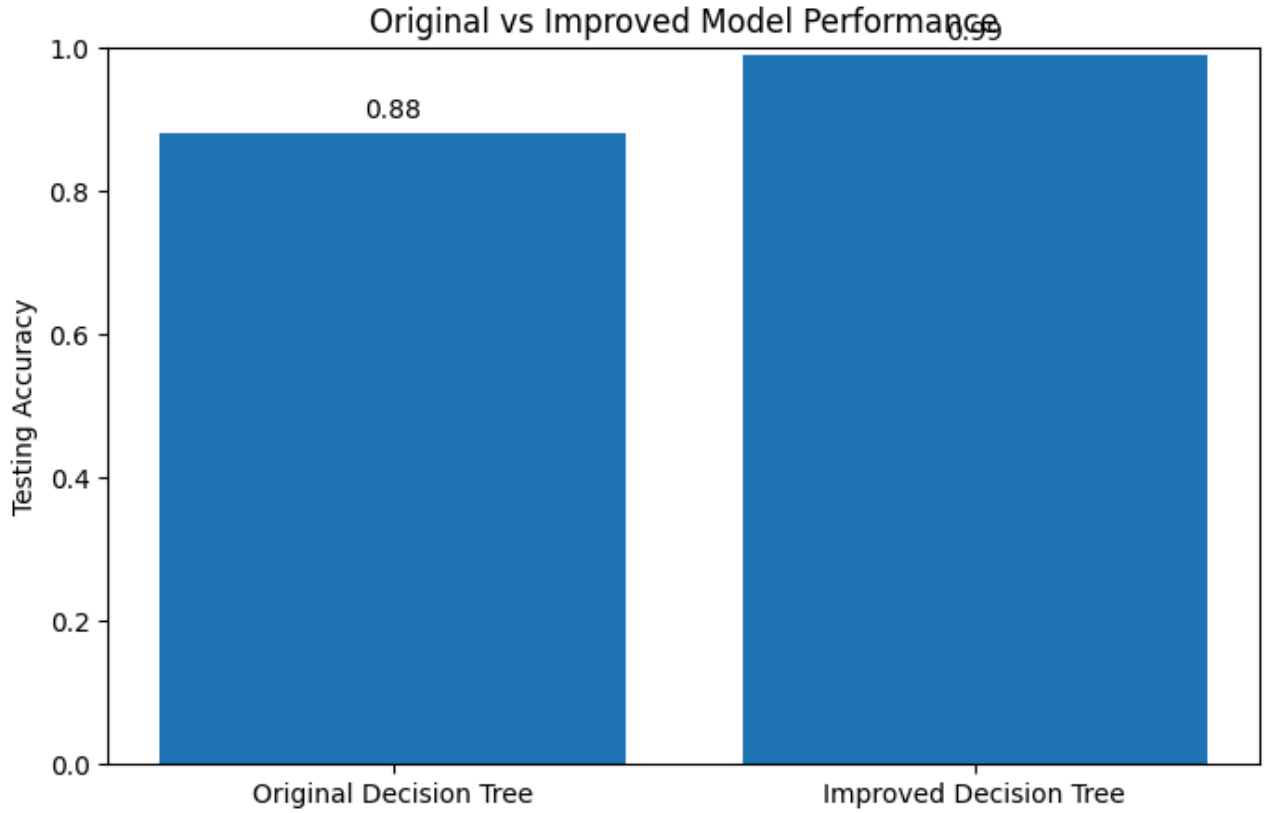
In [57…
```
best_model = grid_search.best_estimator_

best_pred = best_model.predict(X_test)

best_accuracy = accuracy_score(y_test, best_pred)

print("Improved Test Accuracy:", best_accuracy)
```

```
Improved Test Accuracy: 0.9905660377358491
```

In [58…
```
from sklearn.metrics import accuracy_score

original_accuracy = accuracy_score(y_test, y_pred)
improved_accuracy = accuracy_score(y_test, best_pred)

print("Original Model Accuracy :", original_accuracy)
print("Improved Model Accuracy :", improved_accuracy)

if improved_accuracy > original_accuracy:
    print("The improved model performs better.")
elif improved_accuracy < original_accuracy:
    print("The original model performs better.")
else:
    print("Both models have the same accuracy.")
```

```
Original Model Accuracy : 0.8820754716981132
Improved Model Accuracy : 0.9905660377358491
The improved model performs better.
```

In [59…
```
import matplotlib.pyplot as plt

models = ["Original Decision Tree", "Improved Decision Tree"]
accuracies = [original_accuracy, improved_accuracy]

plt.figure(figsize=(8, 5))
plt.bar(models, accuracies)

plt.ylim(0, 1)
plt.ylabel("Testing Accuracy")
plt.title("Original vs Improved Model Performance")

for i, value in enumerate(accuracies):
    plt.text(i, value + 0.02, f"{value:.2f}", ha="center")

plt.show()
```



In [60…

```python
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

final_accuracy = accuracy_score(y_test, best_pred)
final_precision = precision_score(
    y_test, best_pred, average='weighted'
)
final_recall = recall_score(
    y_test, best_pred, average='weighted'
)
final_f1 = f1_score(
    y_test, best_pred, average='weighted'
)

print("Final Model Performance")
print("-----------------------")
print("Accuracy :", final_accuracy)
print("Precision:", final_precision)
print("Recall   :", final_recall)
print("F1-Score :", final_f1)
```

```
Final Model Performance
-----------------------
Accuracy : 0.9905660377358491
Precision: 0.9906281032770605
Recall   : 0.9905660377358491
F1-Score : 0.990567864536179
```